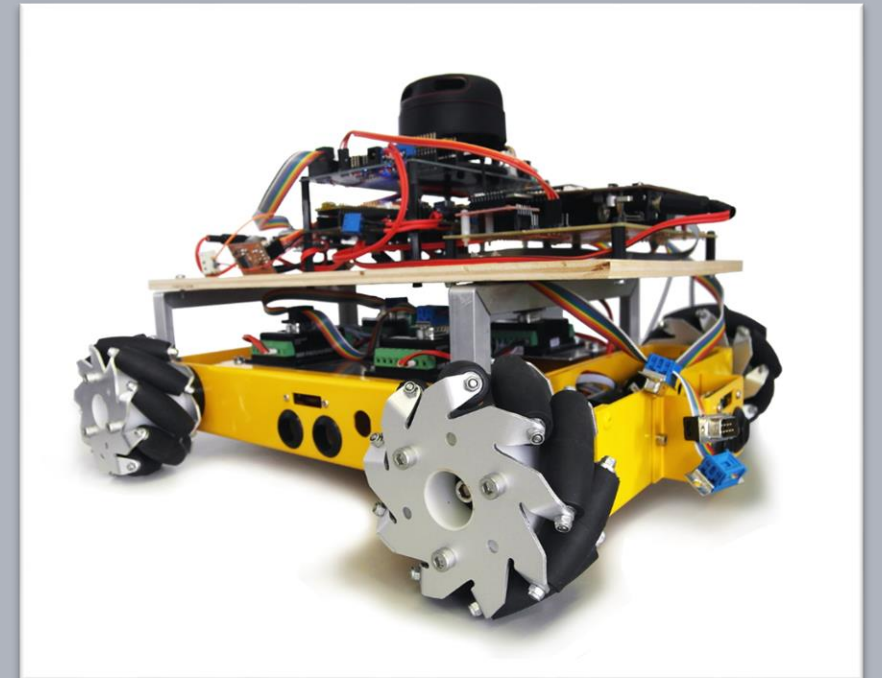


Advanced Embedded Architectures and Applications

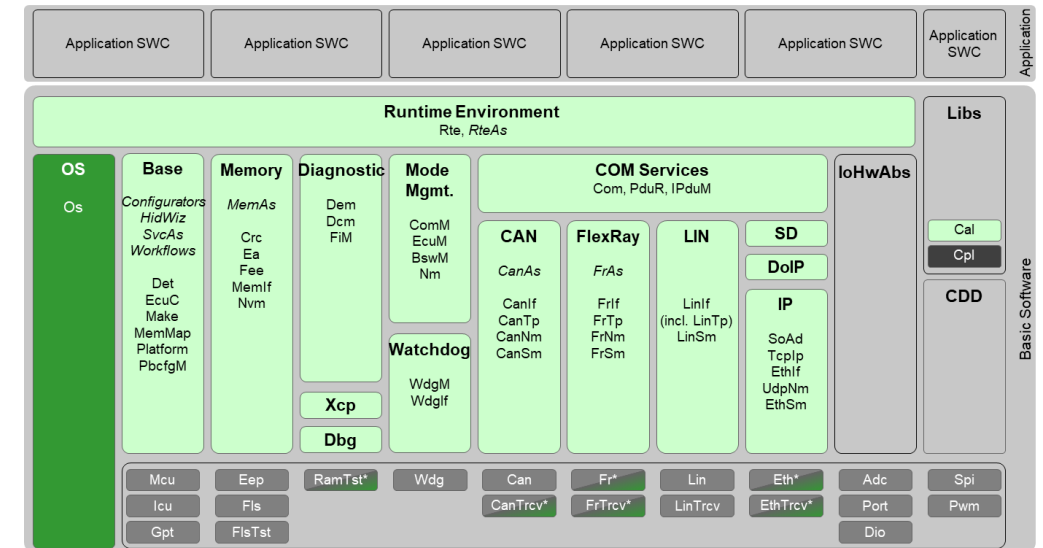
Layered Architecture / AUTOSAR Concept

Prof. Dr.-Ing. Peter Fromm

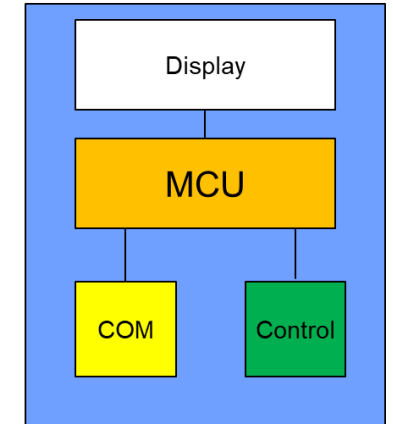
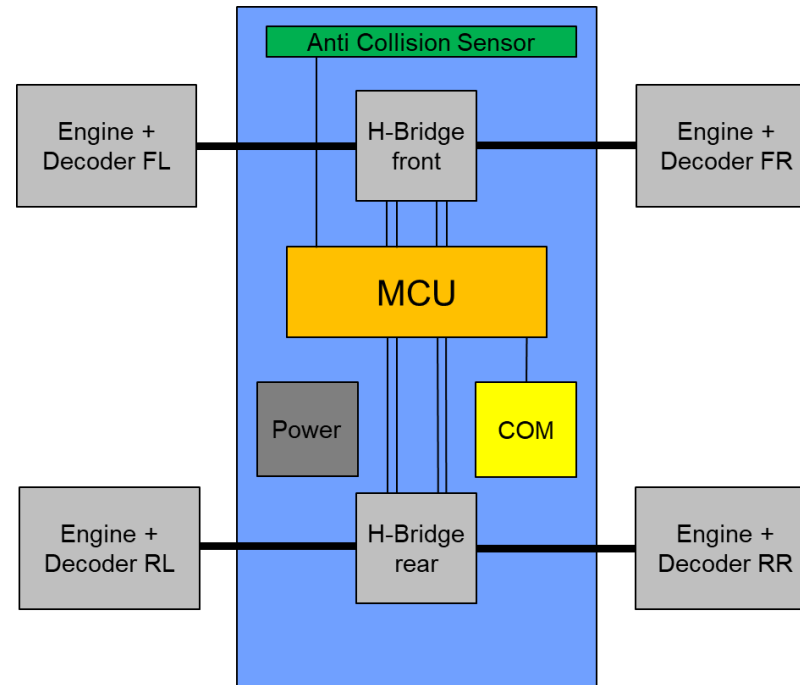
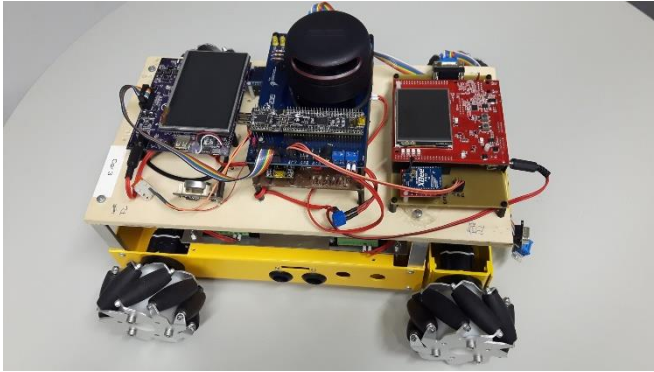


Content

- Motivation
- The runnable concept
- The virtual function bus / RTE
- Synchronous versus asynchronous calls
- The AUTOSAR architecture
- RTE light
- A first framework for our car project



Motivation



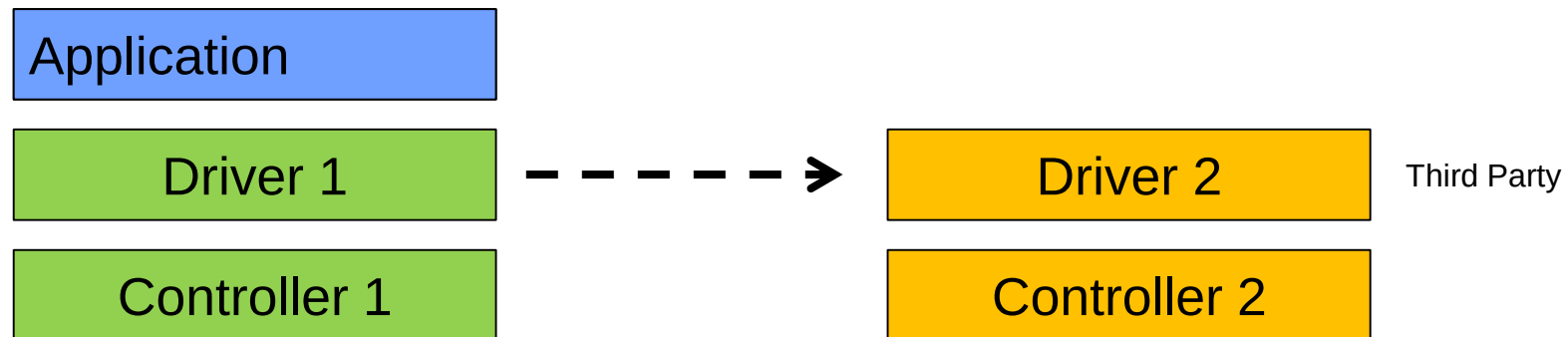
This is what we have

This is what we want to develop

The Dream

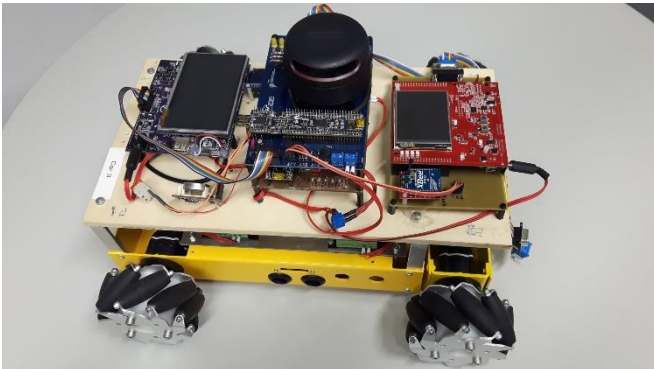
General design concept

- Focus on application development
- Exchange / reuse of components
- Strict separation of driver software (BSW) and application software (SWC)

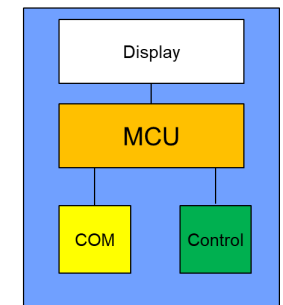
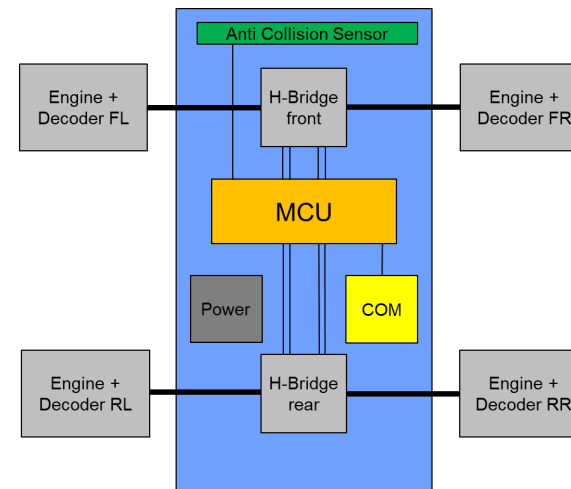


Reality

- Aurix Multicore, 2 x PSOC5LP, FM4
- PXROS, Erika (2 versions)
- Faulhaber Engine Controller
- LIDAR
- CanOpen Bus, UART (Zigbee)
- External Remote Control (PC)

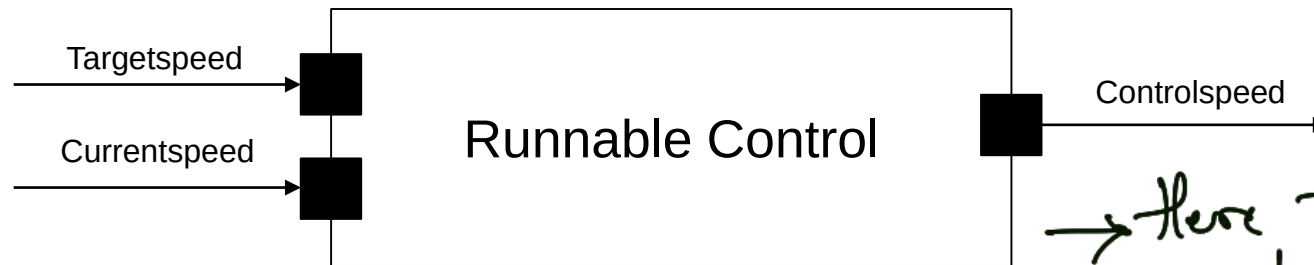


- 2 x PSOC5LP
- Erika
- Engine control by software (H-bridge)
- Ultrasonic or similar
- UART (Zigbee)
- Remote Control (LabBoard)



Reuse Potential

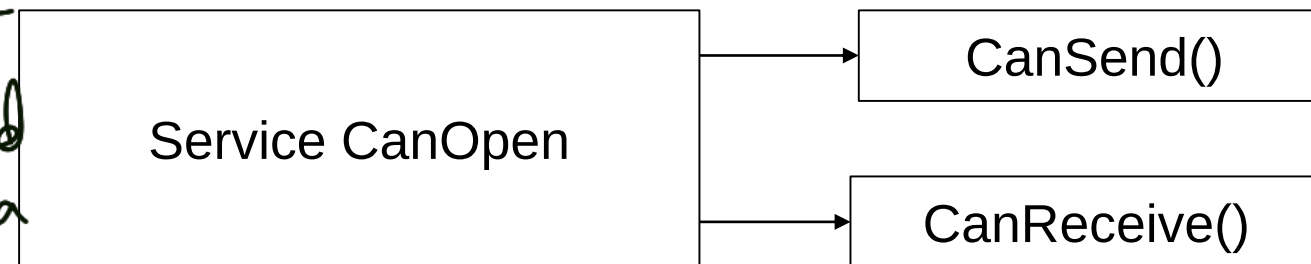
- Application code *Runnable* is a toplevel software function which is called by a RTE
 - As long as it is operational, i.e. gets and provides the correct data / signals



→ here the application is working on a abstract level and not interfacing directly with drivers, it is using a signal driven approach.

- Middleware code (e.g. CanOpen)
 - As long as it has a common API to the drivers

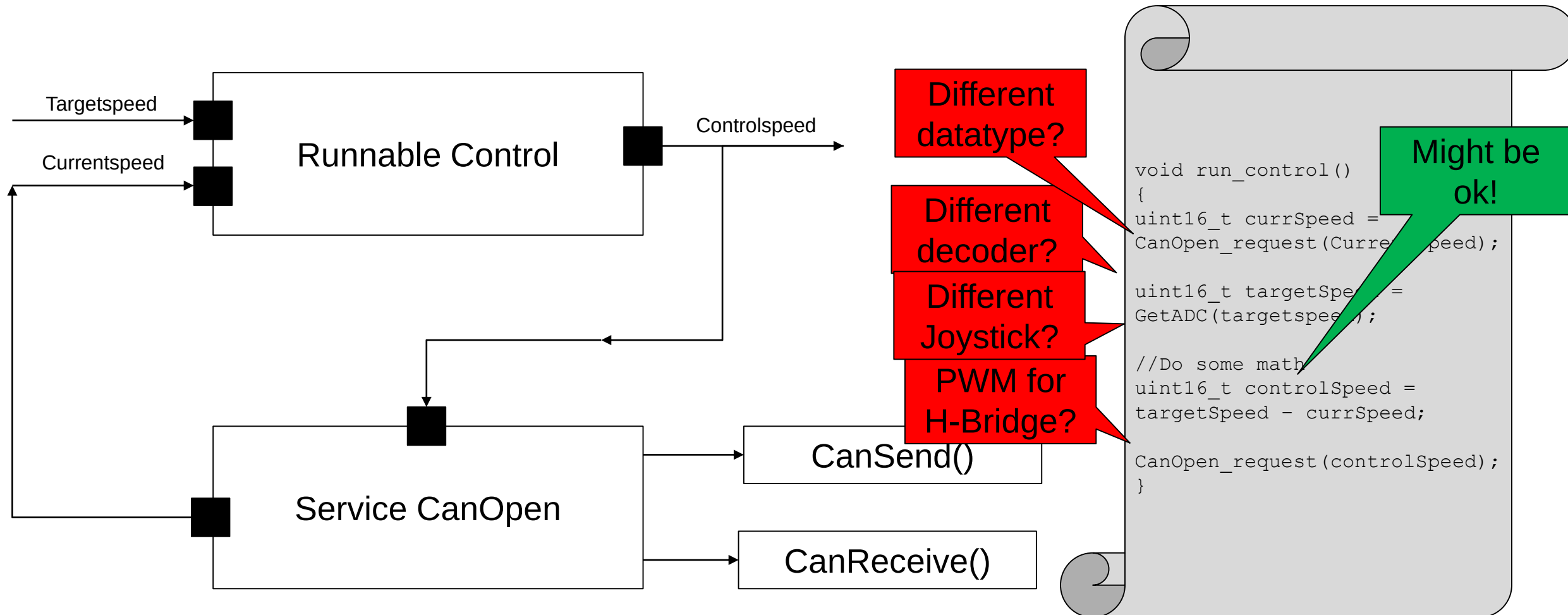
Here we consider data and interfaces bit more abstract
 → Data will be provided via data pool and a common API.



This allows separation of sender and receiver.

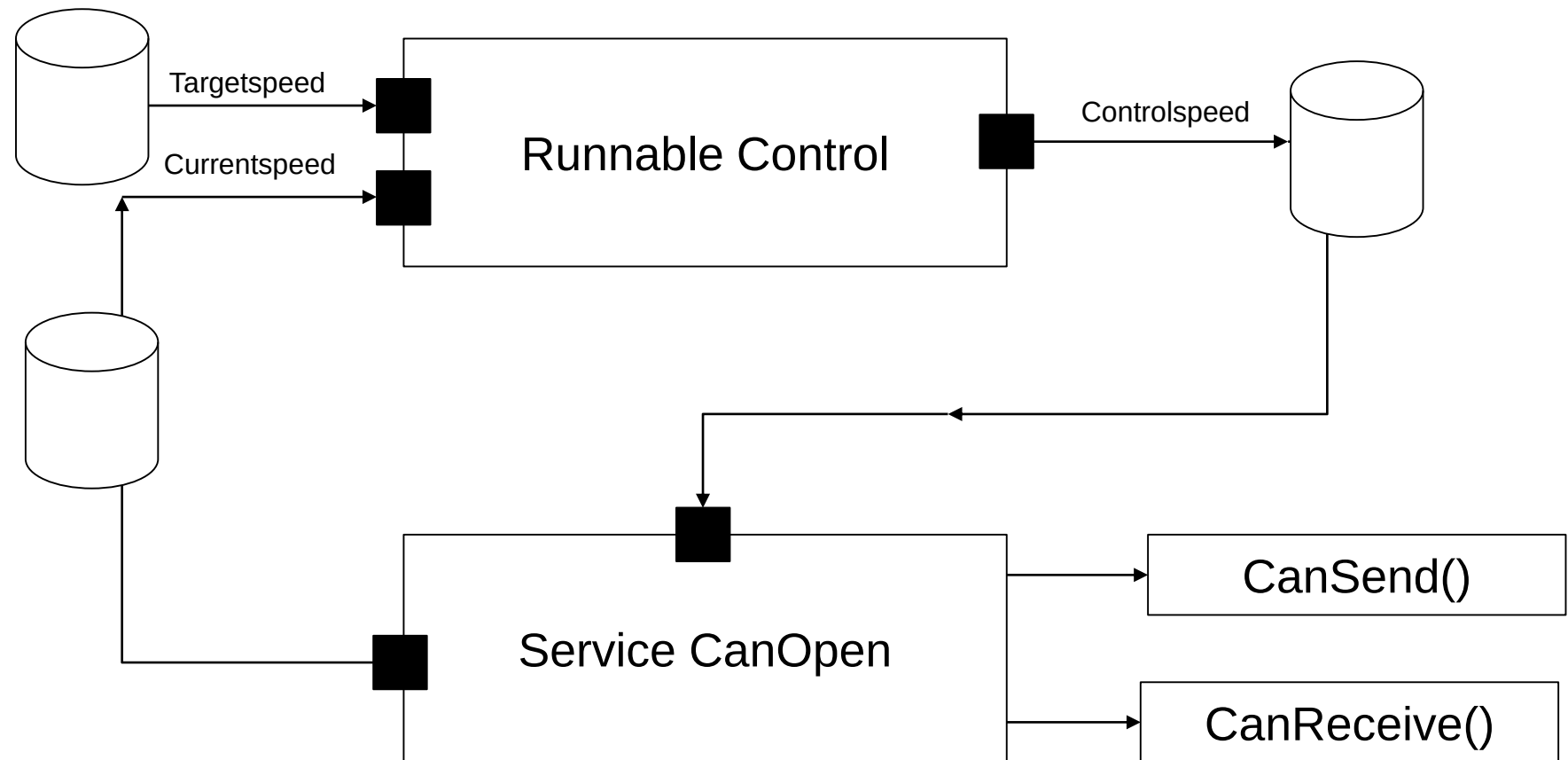
(1) We don't care who sent the data
 (2) We don't care who uses the data.

Translating arrows into code



The runnable / signal concept

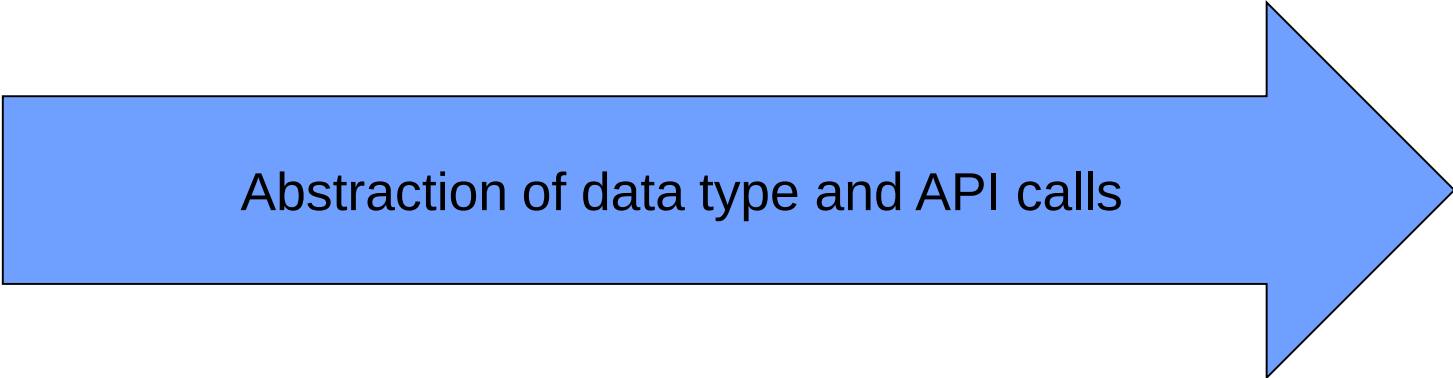
- We have to consider data and interfaces a bit more abstract!
- Data will be provided via a data pool and a common API
- This allows separation of sender and receiver.
- We don't care who provided the data.
- We don't care who uses the data



The signal concept / code

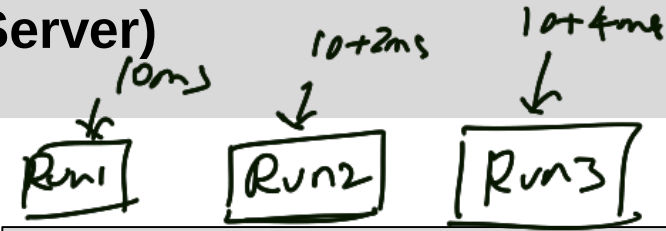
```
void run_control()  
{  
    uint16_t currSpeed =  
    CanOpen_request(CurrentSpeed);  
  
    uint16_t targetSpeed = GetADC(targetspeed);  
  
    //Do some math  
    uint16_t controlSpeed = targetSpeed -  
    currSpeed;  
  
    CanOpen_request(ControlSpeed);  
}
```

```
void run_control()  
{  
    speed_t currSpeed = get_CurrentSpeed();  
  
    speed_t targetSpeed = get_TargetSpeed();  
  
    //Do some math  
    speed_t controlSpeed = targetSpeed - currSpeed;  
  
    set_ControlSpeed(controlSpeed);  
}
```



Abstraction of data type and API calls

Implementation as Synchronous Call (Client/Server)



- Abstracted getter/setter acts as wrapper
- Good for direct driver or service calls
- Clear and easy to understand call sequence
- Low risk for races and critical sections
- Inline can be used for better performance
- Code can be generated out of model
- No support of 1:m communication
- No solution for inter-runnable communication
- Limited to single task context

if there are many consumers, who needs the value then the data is sent at different times having different

```
speed_t get_TargetSpeed()
{
    speed_t targetSpeed = ADC_read();
    return targetSpeed;
}

speed_t get_CurrentSpeed()
{
    speed_t currSpeed =
        (speed_t)CanOpen_request(CurrentSpeed);
    return currSpeed;
}

void set_ControlSpeed(speed_t speed)
{
    CanOpen_request((uint16_t)controlSpeed);
}
```

Run1 is
synchronous
function call.

Let's compare the three functions? Which one is a good design, which one not?

Implementation as Asynchronous Call (Sender/Receiver)

We store the value in a global variable to make it constant for certain time

speed_t globalTargetSpeed = 0; → A global variable to store data.

- Abstracted getter/setter stores data in global buffer and (optionally) calls driver
- Events are used to signal update

- Very flexible
- Semaphores can be used to protect complex structures
- Code can be generated out of model
- Support of 1:m communication
- Solution for inter-runnable communication
- Multi task context supported

- Performance penalty
- More complex API
- Higher risk of races

```
void refresh_TargetSpeed() //Peripheral Interface
{
    GetRessource(res_TargetSpeed);
    globalTargetSpeed = ADC_read();
    ReleaseRessource(res_TargetSpeed);

    SetEvent(tsk_receiver, ev_TargetSpeedUpdate);
}

speed_t get_TargetSpeed() //application read
{
    speed_t targetSpeed = 0;

    GetRessource(res_TargetSpeed);
    targetSpeed = globalTargetSpeed;
    ReleaseRessource(res_TargetSpeed);

    return targetSpeed;
}

void set_TargetSpeed(speed_t speed) //application write
{
    GetRessource(res_TargetSpeed);
    globalTargetSpeed = speed;
    ReleaseRessource(res_TargetSpeed);

    SetEvent(tsk_receiver, ev_TargetSpeedUpdate);
}
```

Asynchronous CanOpen Call

- This is a bit more complex, as we have to split the request and reception of data
- A good pattern is to provide a handler, which is being called by the RX-ISR
- And this handler informs the RTE by an event
- Application code becomes more complex, as we have to implement an asynchronous sender / receiver logic
- Which means two runnables, one for requesting the data and one for processing the data

```
RC_t refresh_CurrentSpeed()
{
    //This is a non blocking call,
    //which returns through the handler

    RC_t result =
    (speed_t)CanOpen_request(CurrentSpeed, handler);

    return result;
}

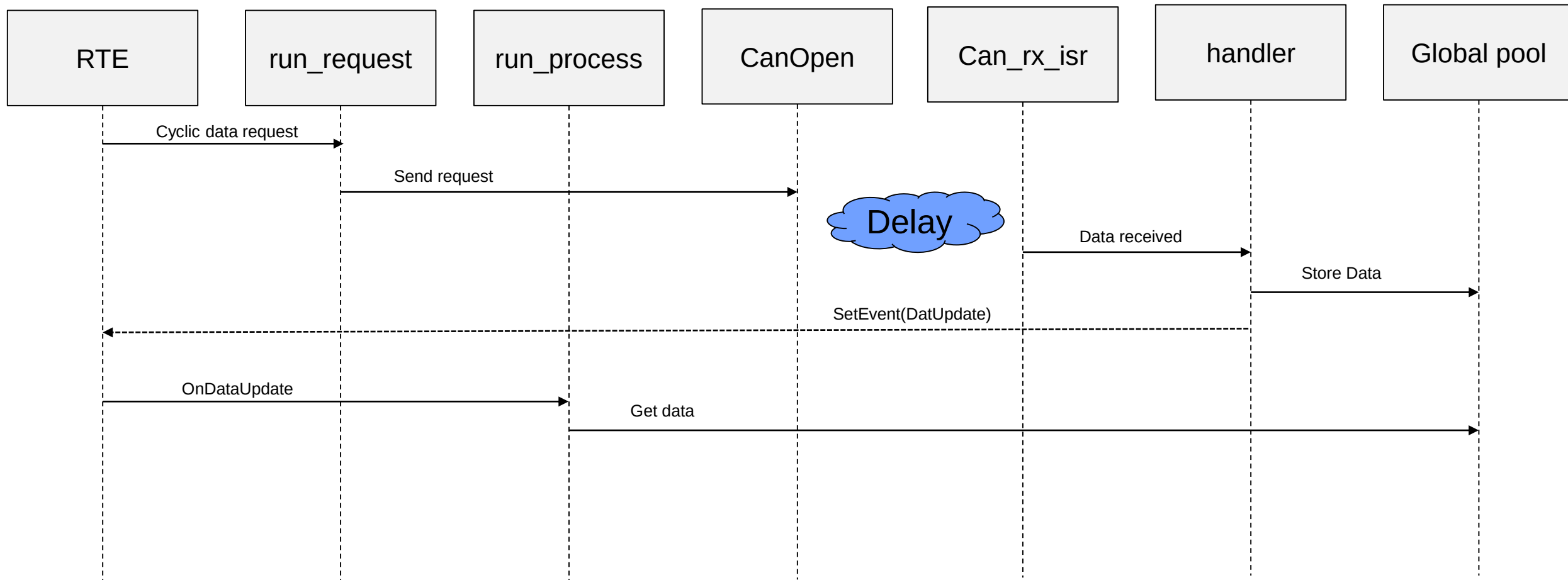
void handler(can_package)
{
    //This handler is called when CanOpen returns

    //Add resource if needed
    globalCurrentSpeed = can_package.currentSpeed;

    SetEvent(tsk_receiver, ev_CurrentSpeedUpdate);
}

speed_t get_CurrentSpeed()
{
    //Add resource if needed
    speed_t currSpeed = globalCurrentSpeed;
    return currSpeed;
}
```

Asynchronous CanOpen Call



Alternative solution – parameter references

```
void run_control()  
{  
    speed_t currSpeed = get_CurrentSpeed();  
  
    speed_t targetSpeed = get_TargetSpeed();  
  
    //Do some math  
    speed_t controlSpeed = targetSpeed - currSpeed;  
  
    set_ControlSpeed(controlSpeed);  
}
```

```
void run_control(speed_t const& currSpeed,  
                speed_t const& targetSpeed,  
                speed_t & controlSpeed)  
{  
  
    //Do some math  
    controlSpeed = targetSpeed - currSpeed;  
}
```

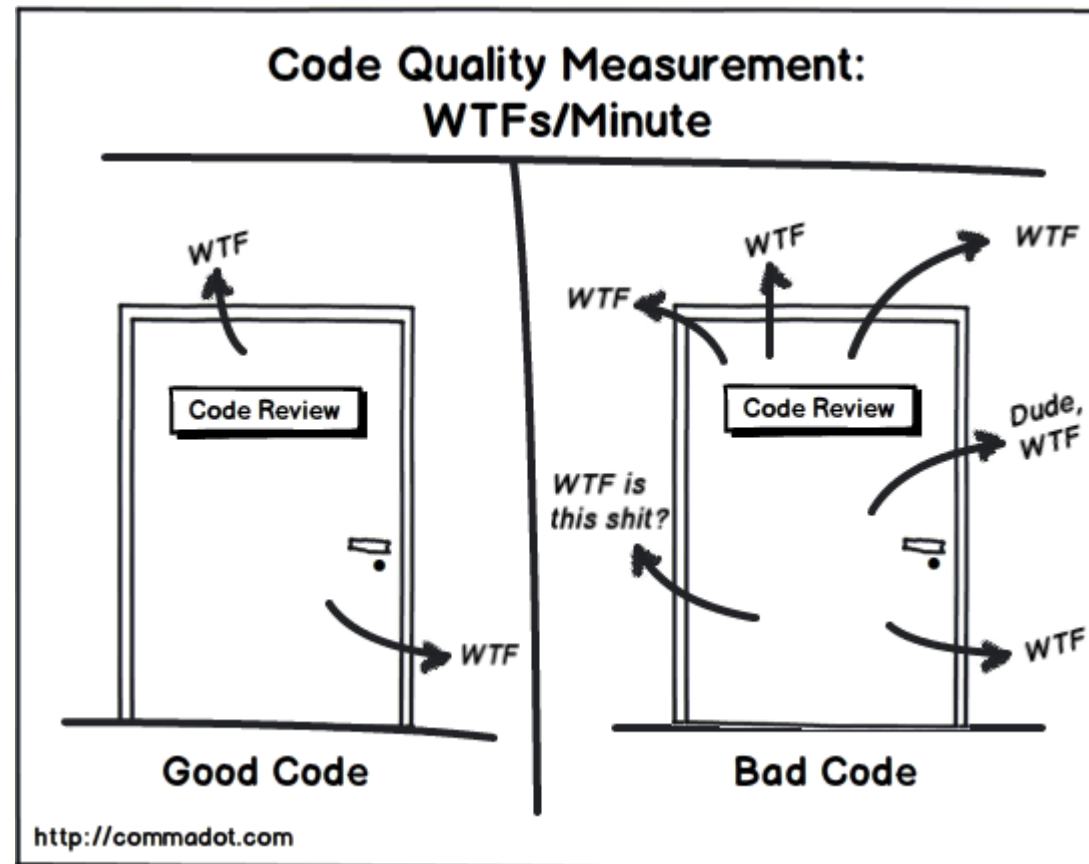


Shift responsibility into Runtime Environment

Comparison of the coding patterns

	Direct driver call	RTE synchronous	RTE asynchronous	Reference parameters
Reuse	-	+	+	+
Performance	+	+	0	+
Extendibility	+	+	+	-
Inter-Task communication	-	-	+	+
1:m communication	-	-	+	+
Model/Code sync	-	0	0	+

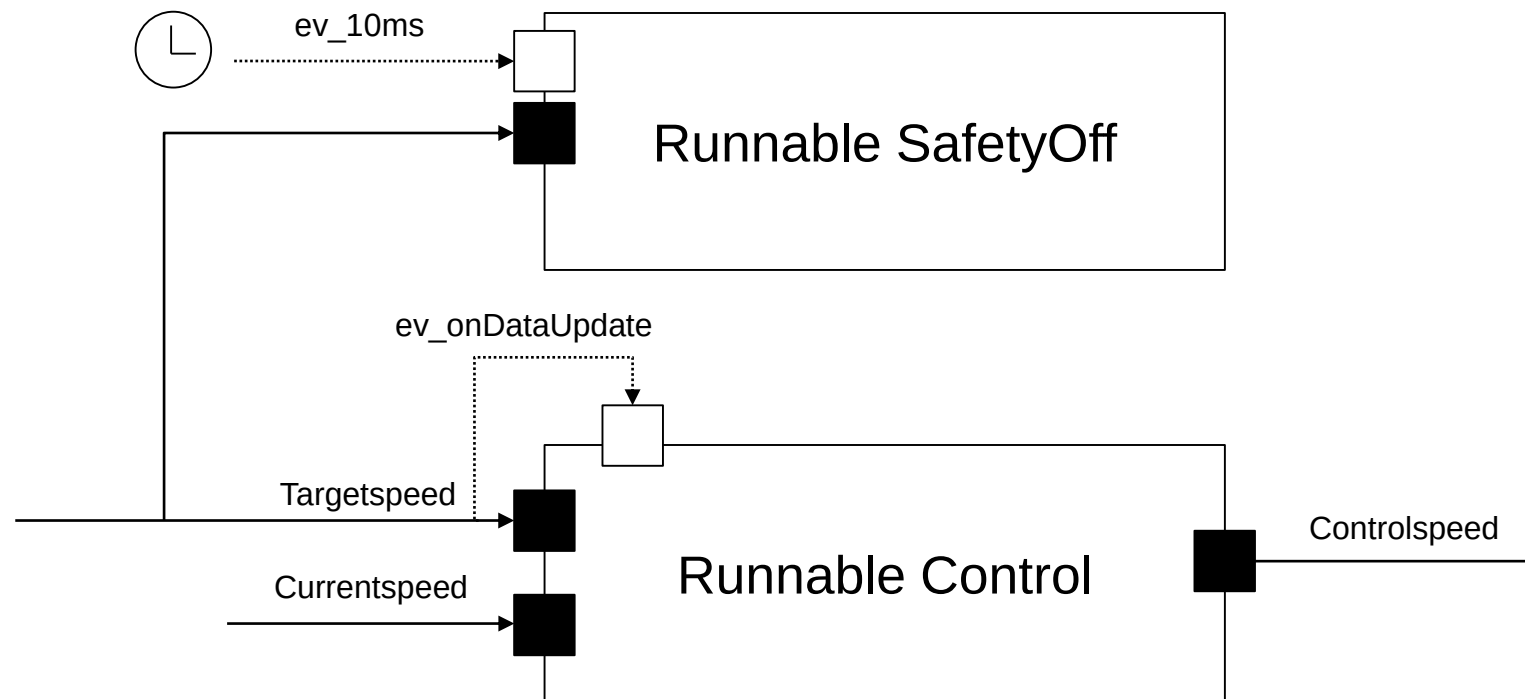
Sidenote: Why model/code synchronization is important



Trigger

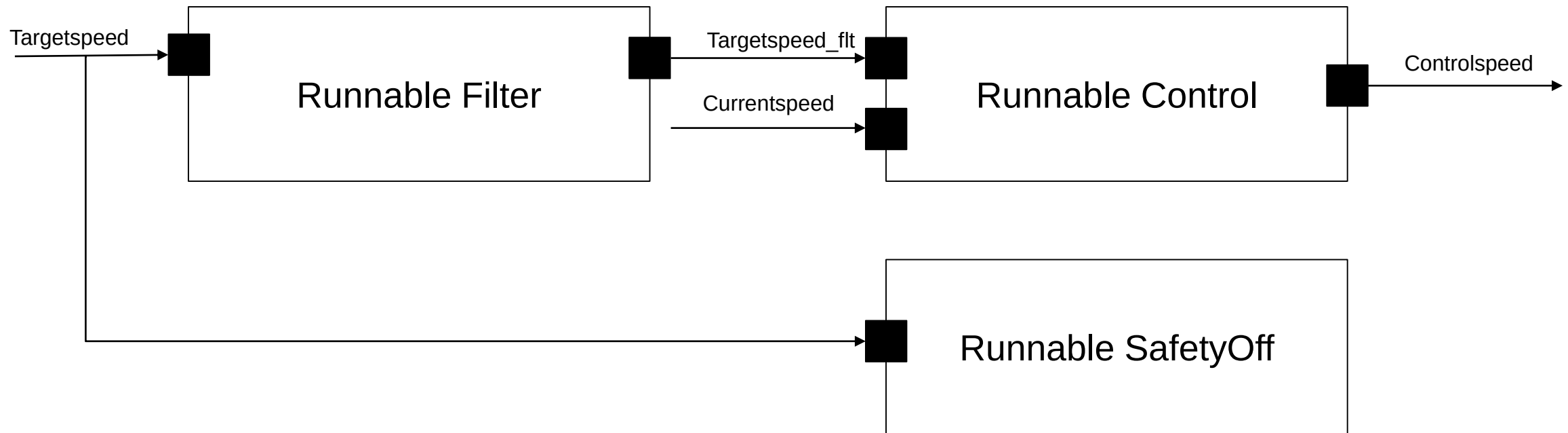
Let's make the use case a bit more complex:

- A safety monitor, which checks for a broken hardware connection shall validate the target value and age and will turn the system off if required
- When are the runnables executed? Same task? Do we have to consider a possible race?



Is the filter a runnable?

Not a good design, because the filter will only be called from the control runnable. Better: Simple function call in the application code.



Runnable definition

- A Runnable Entity is a part of a Software Component which can be executed and scheduled independently from the other Runnables.
- A Runnable is implemented as a C-functions.
- Runnables are executed in the context of a task.
- Runnables are only triggered by the RTE. One runnable is not allowed to call another runnable directly.
- Several Runnables can be executed concurrently, by mapping them to different Tasks.
- Runnables exchange data only via the virtual function bus / RTE.

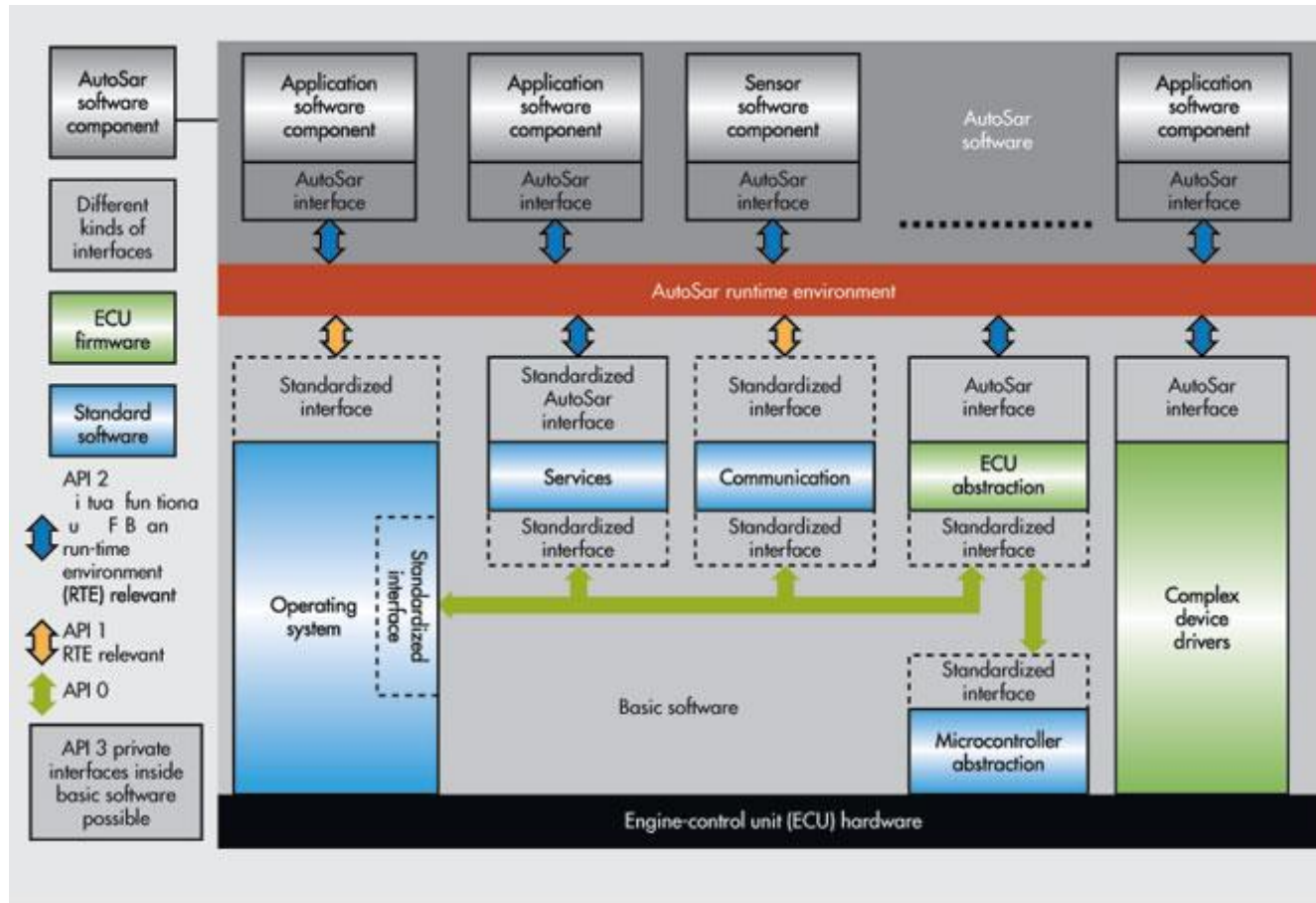
Runnables form the reusable lego bricks for our application.

AUTOSAR

AUTOSAR (AUTomotive Open System ARchitecture) is an open and standardized automotive software architecture, jointly developed by automobile manufacturers, suppliers and tool developers. It is a partnership founded 2003 of automotive OEMs, suppliers and tool vendors whose objective is to create and establish open standards for automotive E/E (Electrics/Electronics) architectures that will provide a basic infrastructure to assist with developing vehicular software, user interfaces and management for all application domains.

AUTOSAR follows the layered architecture concept. The total software solution is divided into 3 layers, known as Application software, Runtime Environment (RTE) and Basic Software (BSW). All the functionalities are kept under the application software module, the Run Time Environment acts as a virtual function bus for an Electronic control unit and messages are transferred using the basic software.

Autosar Architecture

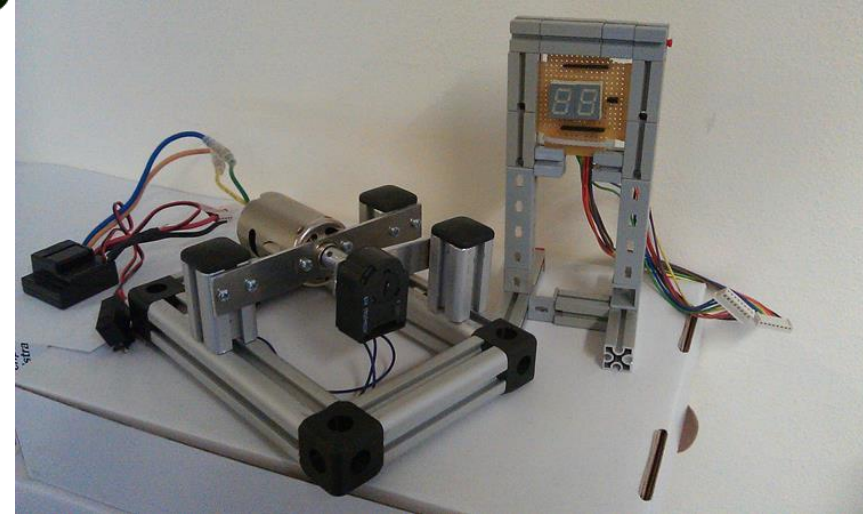


The Autosar architecture covers many of the protocols necessary for automotive Infotainment.

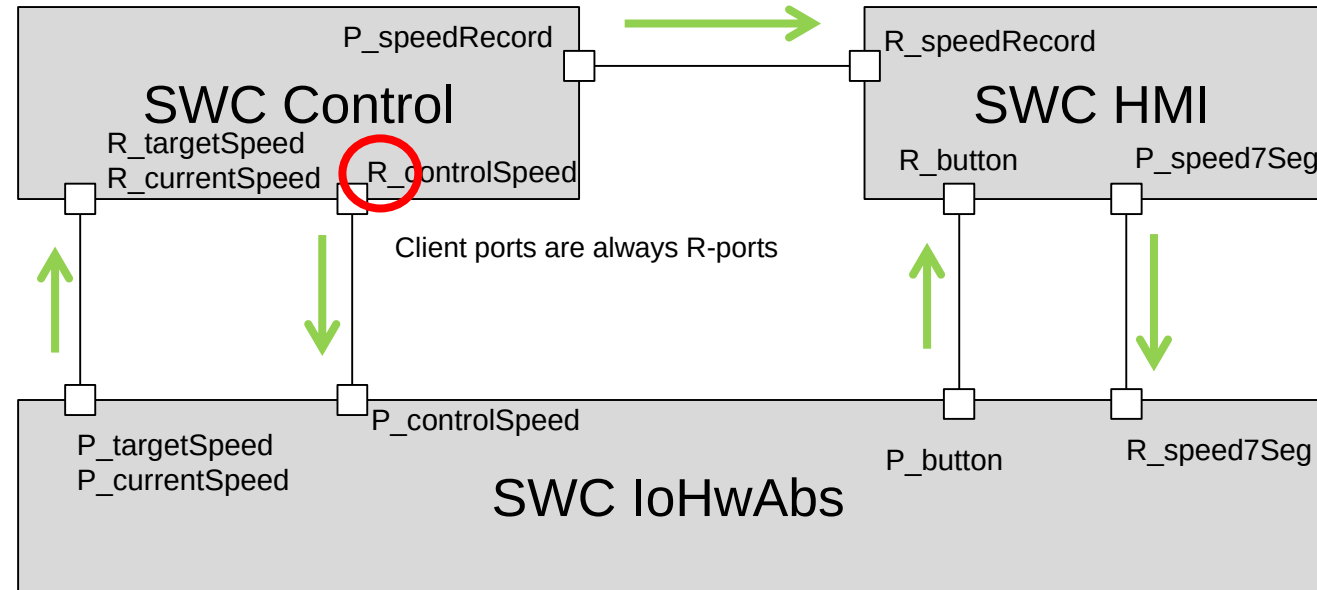
Example – Control System

- Target Speed is read from a potentiometer
- Current speed is read from an incremental decoder
- Control speed is calculated by controller
- The different parameters can be displayed on a 7-segment display
- A button will be used to toggle between the different values to be displayed

*Target Speed – PWM
Current Speed – Decoder.*



A first high level design



Require Ports require a service or data
Provide port provide a service or data

Interface types

	Client / Server	Sender / Receiver
Type	Synchronous	Asynchronous
Implementation	Function call	- Copy data to central buffer - Fire event
Features	- Un-queued - Queued	- Timeout - Resync
Symbol		
Execution order	- Explicit (event is fired directly after data transmission) - Implicit (event is fired when all data for a runnable is transferred)	

Data Types

Autosar provides several levels of abstraction for data types:

Application data types

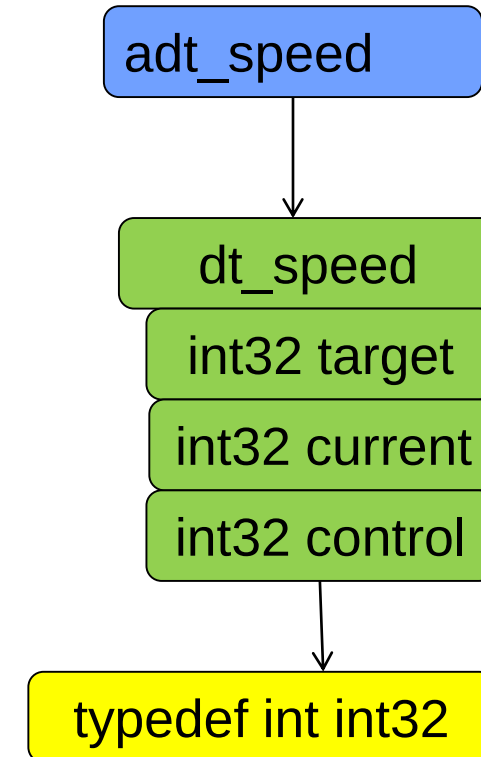
- Used for the application design
- Implementation independent

Implementation data types

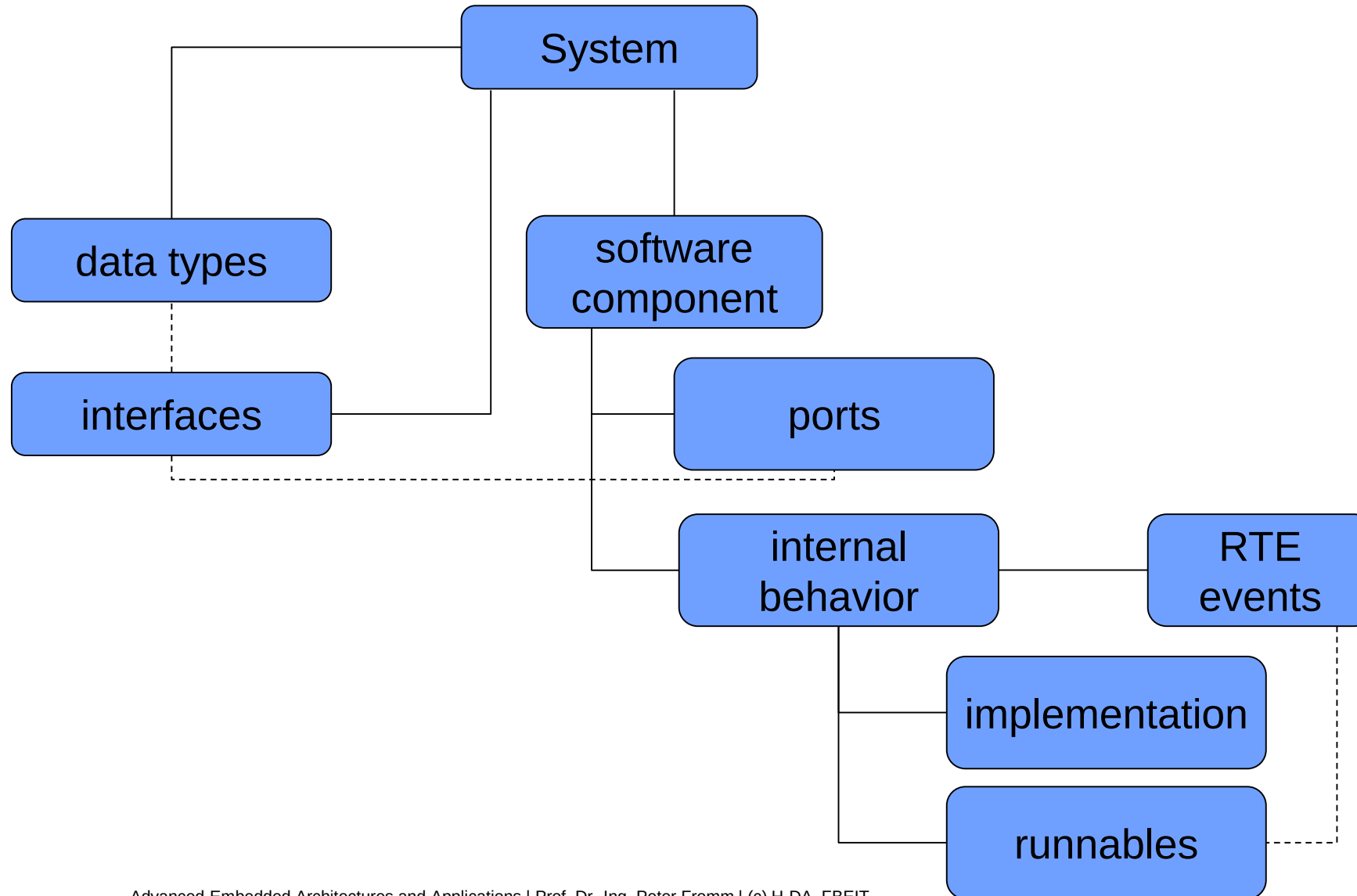
- Used for the implementation
- Application data types have to be mapped to implementation data types

Machine data types

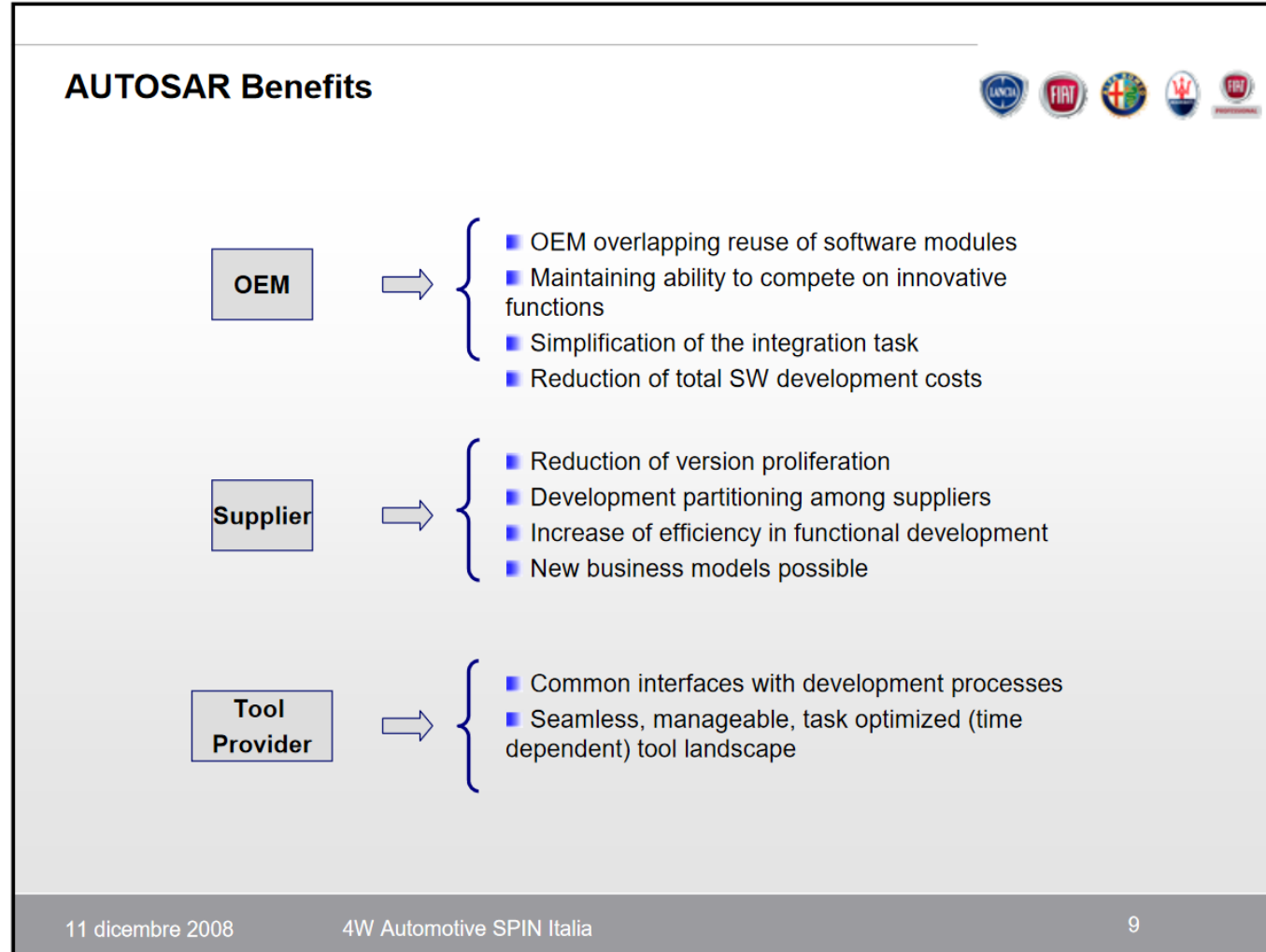
- Atomic data types like sint32, int32 are mapped to compiler data types
- like int, unsigned int



Autosar Meta Model



AUTOSAR Marketing



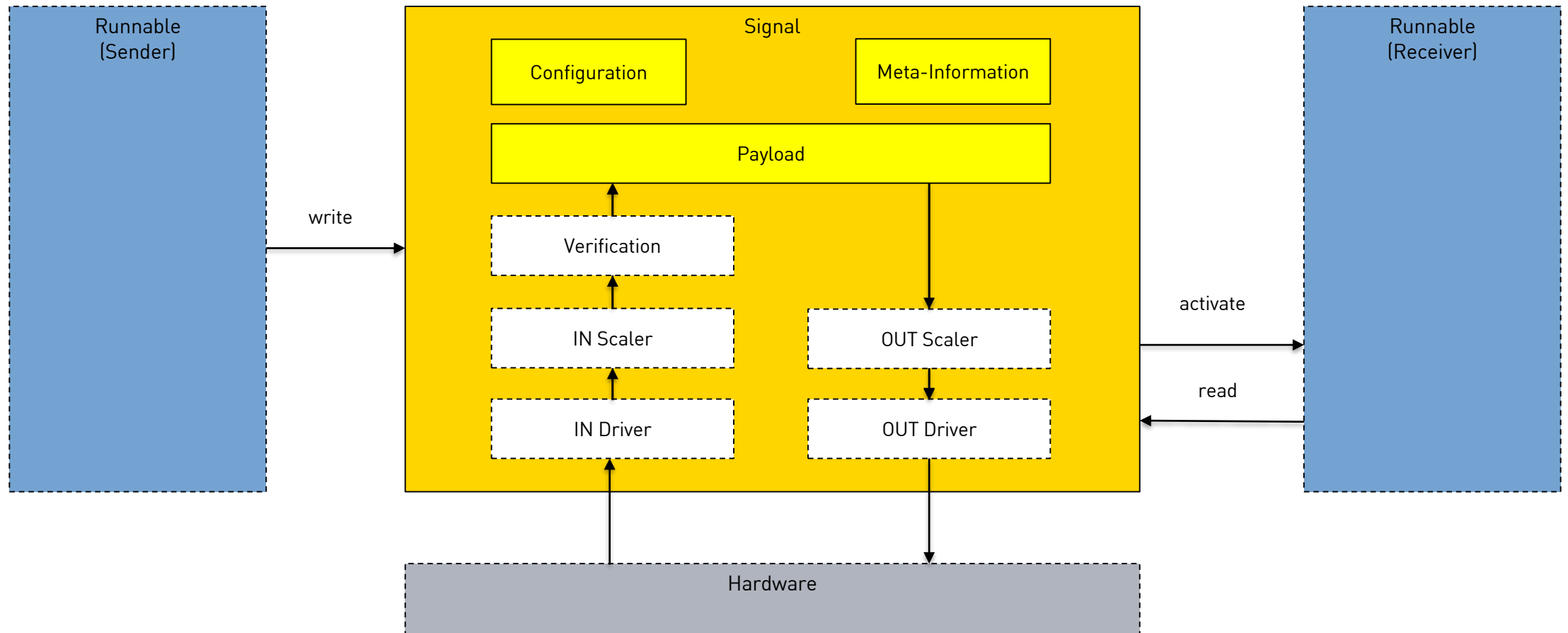
Source: <http://www.automotive-spin.it/uploads/4/mitidieri-4W.pdf>

The concept is good, but the implementation is too complex and too limiting at the same time.

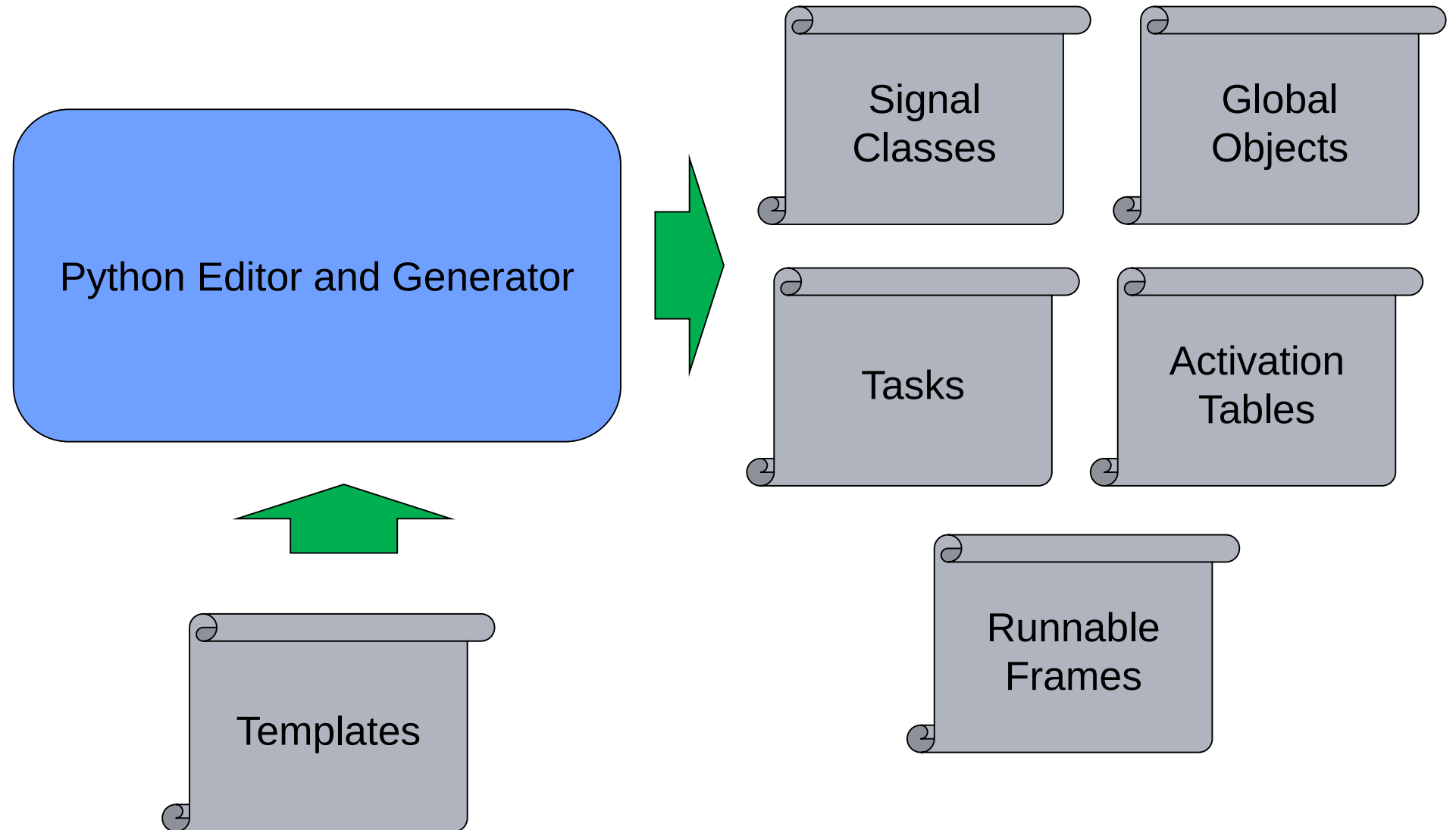
Many stakeholders worked on the specification!

Source: My opinion

Own Solution: Signals and Runnables – Big Picture

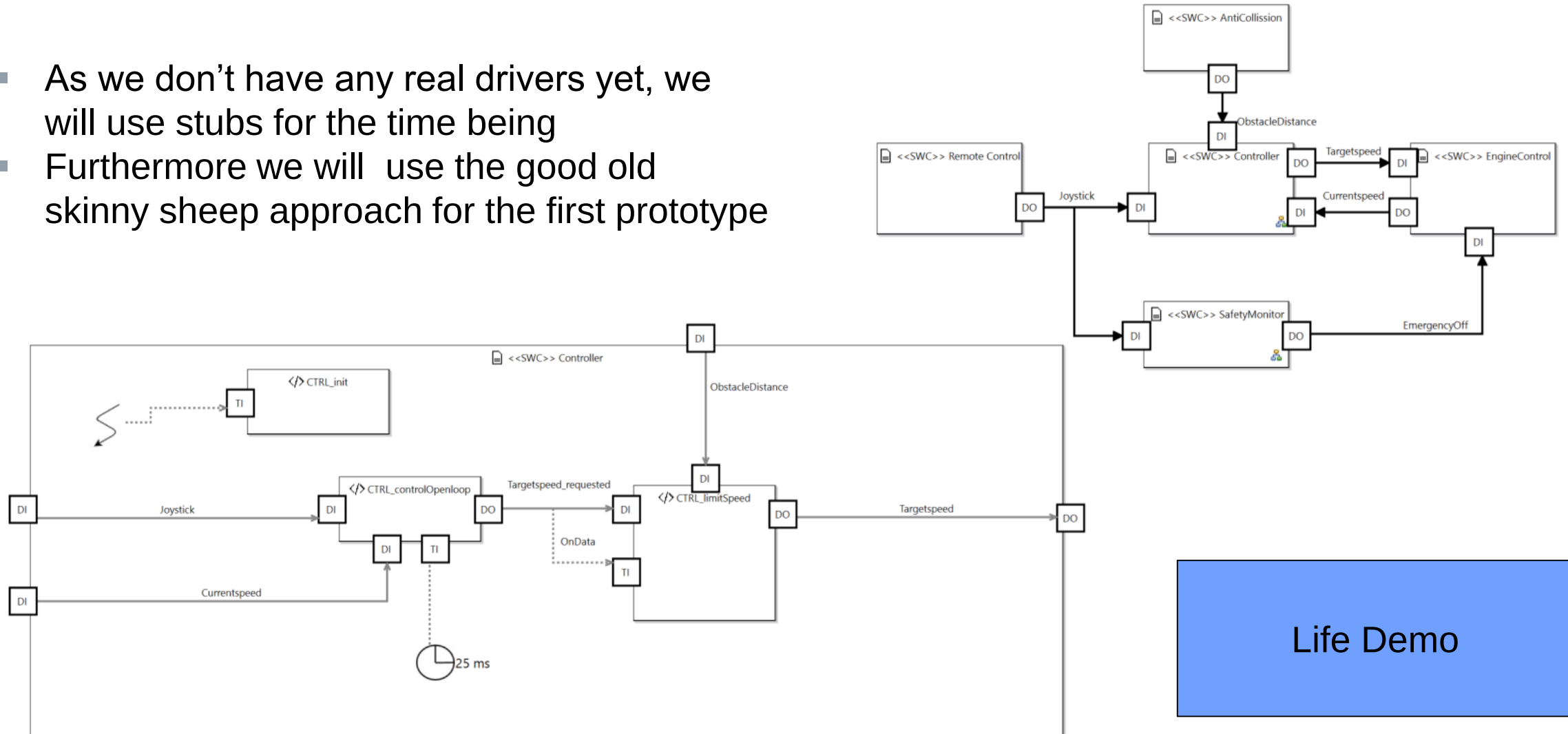


Our RTE Light Generator



Let's create a first framework for our car

- As we don't have any real drivers yet, we will use stubs for the time being
- Furthermore we will use the good old skinny sheep approach for the first prototype



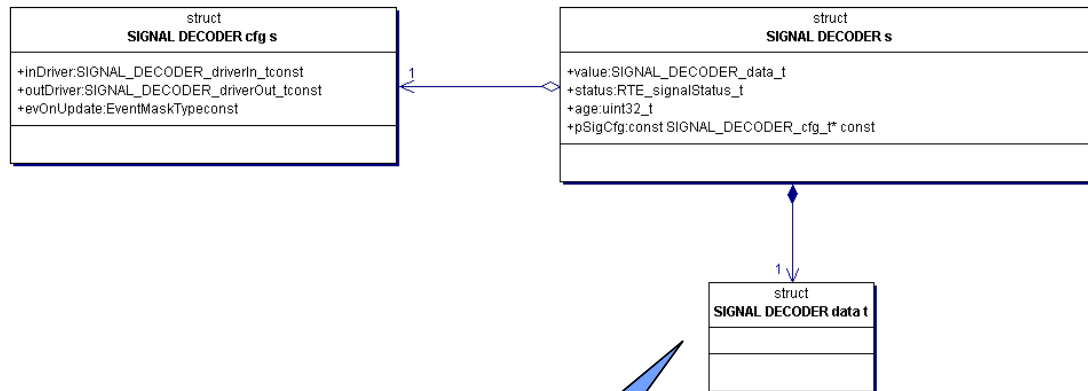
Life Demo

Generated Signals

Struct

```

{
    SIGNAL_DECODER_driverIn_t const inDriver;
    SIGNAL_DECODER_driverOut_t const outDriver;
    EventMaskType const evOnUpdate;
} SIGNAL_DECODER_cfg_s;
    
```



Must be added
by user

Struct
 {
 SIGNAL_DECODER_data_t value;
 RTE_signalStatus_t status;
 uint32_t age;
 SIGNAL_DECODER_cfg_t *const pSigCfg;
 } SIGNAL_DECODER_s;

```

Include directives:
- global.h
- project.h
- rte.h
- Signal_Decoder_type.h

Macros:
- SIGNAL_DECODER_SIGNAL_H_INCLUDED
- SIGNAL_DECODER_DATATRIGGER

Structures:
- SIGNAL_DECODER_cfg_s
    + inDriver: const SIGNAL_DECODER_driverIn_t
    + outDriver: const SIGNAL_DECODER_driverOut_t
    + evOnUpdate: const EventMaskType
- SIGNAL_DECODER_s
    + value: SIGNAL_DECODER_data_t
    + status: RTE_signalStatus_t
    + age: uint32_t
    + pSigCfg: const SIGNAL_DECODER_cfg_t *const

Typedefs:
- SIGNAL_DECODER_driverIn_t: enum eRC (*) (SIGNAL_DECODER_data_t *)
- SIGNAL_DECODER_driverOut_t: enum eRC (*) (const SIGNAL_DECODER_data_t *)
- SIGNAL_DECODER_cfg_t: const struct SIGNAL_DECODER_cfg_s
- SIGNAL_DECODER_t: struct SIGNAL_DECODER_s

Function definitions:
- RTE_SIGNAL_DECODER_init(SIGNAL_DECODER_t *const, const SIGNAL_DECODER_data_t *const): RC_t
    + signal: SIGNAL_DECODER_t *const
    + value: const SIGNAL_DECODER_data_t *const
    + ret: RC_t
- RTE_SIGNAL_DECODER_getStatus(const SIGNAL_DECODER_t *const): RTE_signalStatus_t
    + signal: const SIGNAL_DECODER_t *const
- RTE_SIGNAL_DECODER_setStatus(SIGNAL_DECODER_t *const, const RTE_signalStatus_t): RC_t
    + signal: SIGNAL_DECODER_t *const
    + status: const RTE_signalStatus_t
- RTE_SIGNAL_DECODER_getAge(const SIGNAL_DECODER_t *const): uint32_t
    + signal: const SIGNAL_DECODER_t *const
- RTE_SIGNAL_DECODER_incAge(SIGNAL_DECODER_t *const, uint32_t): void
    + signal: SIGNAL_DECODER_t *const
    + increment: uint32_t
- RTE_SIGNAL_DECODER_get(const SIGNAL_DECODER_t *const): SIGNAL_DECODER_data_t
    + signal: const SIGNAL_DECODER_t *const
- RTE_SIGNAL_DECODER_set(SIGNAL_DECODER_t *const, const SIGNAL_DECODER_data_t): RC_t
    + signal: SIGNAL_DECODER_t *const
    + value: const SIGNAL_DECODER_data_t
    + ret: RC_t
- RTE_SIGNAL_DECODER_pullPort(SIGNAL_DECODER_t *const): RC_t
    + signal: SIGNAL_DECODER_t *const
    + ret: RC_t
    + value: SIGNAL_DECODER_data_t
- RTE_SIGNAL_DECODER_pushPort(const SIGNAL_DECODER_t *const): RC_t
    + signal: const SIGNAL_DECODER_t *const
    + ret: RC_t
    
```

Signal API

API	Description	
Signal_init()	Initializes the signal.	
Signal_getStatus()	Check the Validity Status of the Signal.	Error handling
Signal_setStatus()	Set the Validity Status of the Signal.	
Signal_getAge()	Get the time in ms since the last Update.	
Signal_incAge()	Increment the age. This function is called by the RTE only.	
Signal_get()	Get the Payload from the Signal.	Interrunnable Communication
Signal_set()	Write the payload to the signal	
Signal_pullPort()	Update the Signal Value by calling the connected Hardware Driver.	Hardware Access
Signal_pushPort()	Write the current Value to the connected Hardware Driver.	

Activation Engine (slightly extended in the latest release)

```
TASK(tsk_EventDispatcher)
{
    EventMaskType ev = 0;

    while(1)
    {
        //Wait, read and clear the event
        WaitEvent(RTE_EVENTMASK);
        GetEvent(tsk_EventDispatcher, &ev);
        ClearEvent(ev);

        //Process all events from the mask
        RTE_ProcessEventTable(RTE_eventActivationTable,
            RTE_eventActivation_size, ev);
    }
}

TASK(tsk_CyclicDispatcher)
{
    //Increment the age of all signals by 1ms
    RTE_timertick();

    //Process all cyclic runnables which are due
    RTE_ProcessCyclicTable(RTE_cyclicActivationTable,
        RTE_cyclicActivation_size, 1);
}
```

/* Cyclic Table */

```
const RTE_cyclicTable_t RTE_cyclicActivationTable[] = {
    { CONTROL_calculateTargetspeed_run, 25 },
    { REMOTE_readProtocol_run, 25 },
    { ENGINE_getDecoder_run, 100 },
};
```

```
const uint16_t RTE_cyclicActivation_size = sizeof(RTE_cyclicActivationTable) /
    sizeof(RTE_cyclicTable_t);
```

/* Event Table */

```
const RTE_eventTable_t RTE_eventActivationTable[] = {
    { CONTROL_limitSpeed_run, ev_targetspeedRequested_onUpdate },
    { ENGINE_setSpeed_run, ev_targetspeed_OnUpdate },
};
```

```
const uint16_t RTE_eventActivation_size = sizeof(RTE_eventActivationTable) /
    sizeof(RTE_eventTable_t);
```

Software Component (SWC)

SWC cannot call another SWC
SWC is a file containing functions.

```
void CONTROL_calculateTargetspeed_run()
{
    LOG_I("CONTROL_calculateTargetspeed_run", "");

    //Read the joystick data from the RTE
    SIGNAL_JOYSTICK_data_t joystick = RTE_SIGNAL_JOYSTICK_get(&SO_JOYSTICK_signal);

    //Calculate the requested target speed
    SIGNAL_SPEED_data_t targetSpeed_requested = SIGNAL_SPEED_INIT_DATA;

    //The joystick y-value represents the forward speed
    //The joystick x-value represents the car rotation
    targetSpeed_requested.m_vy = CAR_MAXSPEED * (float)joystick.m_y / CAR_MAXJOYSTICK;
    targetSpeed_requested.m_vphi = CAR_MAXROTATION * (float)joystick.m_x / CAR_MAXJOYSTICK;

    //Send the target speed requested signal to the limitSpeed runnable
    RTE_SIGNAL_SPEED_set(&SO_TARGETSPEEDREQUESTED_signal, targetSpeed_requested);
}
```

Input

Logic

Output

Wrap-Up

- Runnables and Signals
- RTE = Signalpool + Runnable Activations
- Realised a first framework for our car

- RTE Light as a very simple framework generator
- Limitations we came across
 - Asynchronous driver implementation is missing
 - No performance optimization (call by value)
 - Only one event and cyclic task (fixed in the new release)
 - No graphical editor (Also available by now ;-)
 - OS dependencies in the generated code (is this really a limitation?)

